

DL Midsem Notes — Section 10 (Extended)

Convolutional Neural Networks, Explained with Diagrams

1 Theory

1.1 Convolution, Intuitively

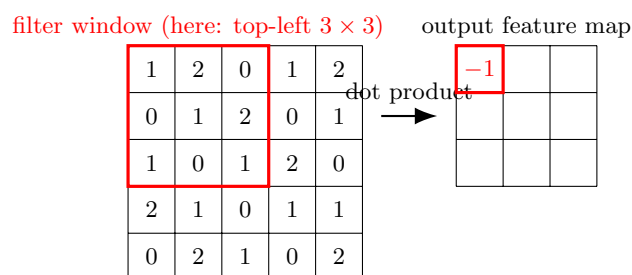
A fully-connected layer looks at the *entire* input at once, with a separate weight for every input pixel. A convolution layer instead looks at a small *local patch* at a time, and reuses the *exact same* small set of weights (the filter) at every location in the image. Two ideas, both driven by how images actually work:

- **Locality:** a pixel's meaning (edge, texture) is determined by its immediate neighbours, not by a pixel on the other side of the image. So look locally.
- **Weight sharing (translation invariance):** an edge detector that works in the top-left corner should work identically in the bottom-right corner. So use the *same* filter everywhere instead of learning a new one per location.

This is also *why* CNNs have so many fewer parameters than a fully-connected layer over the same image — one filter's weights are reused at every spatial position instead of a fresh weight per pixel per position.

1.2 The Sliding Window, Step by Step

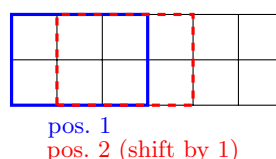
The filter is placed at the top-left of the input, a dot product is computed between the filter and the patch underneath it, and that single number becomes one cell of the output (**feature map**). The filter then slides over by the stride and repeats.



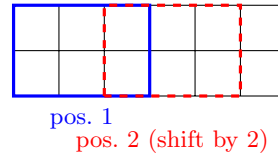
After computing this one output cell, the window slides right by the stride, and the process repeats until the filter has swept the whole image, producing the complete feature map.

1.3 Stride: How Far the Window Jumps

Stride $S = 1$ (windows overlap heavily)



Stride $S = 2$ (less overlap, smaller output)



Bigger stride $\Rightarrow$ fewer positions the filter visits $\Rightarrow$ smaller output feature map, at the cost of skipping some spatial detail.

1.4 Padding: Controlling Edge Behaviour and Output Size

Without padding, pixels near the border are covered by fewer filter positions than central pixels (border information gets used less), and the output shrinks every layer. Padding adds a border of zeros so the filter can be centered on border pixels too:

$P = 1$ zero-padding around a 5×5 input $\rightarrow$ effective 7×7

0	0	0	0	0	0	0
0	1	2	0	1	2	0
0	0	1	2	0	1	0
0	1	0	1	2	0	0
0	2	1	0	1	1	0
0	0	2	1	0	2	0
0	0	0	0	0	0	0

“Valid” convolution = no padding ($P = 0$), output shrinks. **“Same” convolution** = pad just enough that output size equals input size (for stride 1: $P = (F - 1)/2$).

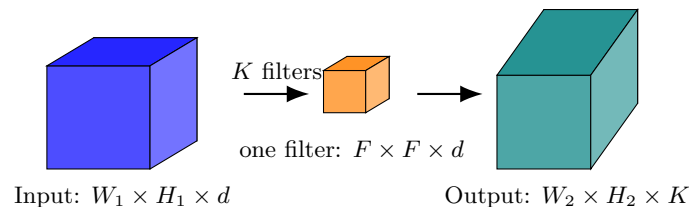
1.5 Output Dimension Formula (from the picture above)

Effective (padded) width is $W_1 + 2P$. The filter (width F) fits at position 0, then every subsequent position uses up S more pixels of that padded width, until it no longer fits:

$$W_2 = \frac{W_1 + 2P - F}{S} + 1, \quad H_2 = \frac{H_1 + 2P - F}{S} + 1$$

1.6 Depth: Multiple Channels and Multiple Filters

A real image has depth d (e.g. $d = 3$ for RGB). A filter must match that depth exactly — it’s really an $F \times F \times d$ block, and its single dot product sums over *all* channels at once, still producing one number per spatial position. Using K different filters (each independently learned) gives K separate feature maps, stacked to form an output of depth K :



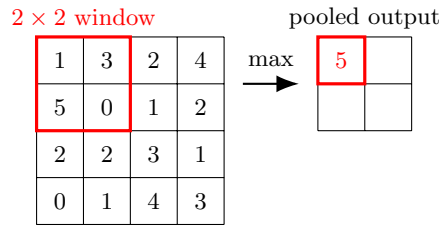
Each of the K filters contributes $F \times F \times d$ weights + 1 bias, and filters don’t share weights with each other, so:

$$\# \text{params} = K(F^2d + 1)$$

Crucially this count has *no* dependence on W_1, H_1 — the same filter is reused at every spatial location, which is exactly the parameter-sharing idea from §1.1.

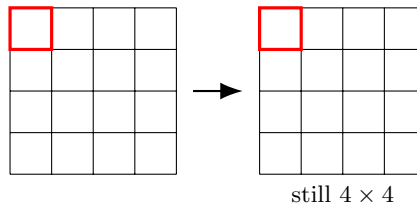
1.7 Pooling: Shrinking Without Learning

Max pooling summarizes each small window to its maximum value — no weights, no learning, just a fixed downsampling rule that also adds mild translation tolerance (shifting the input by one pixel barely changes which value was the max of a window).

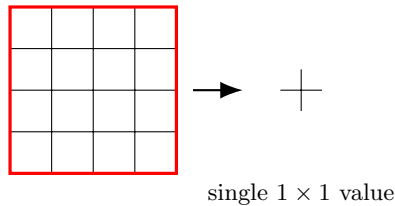


1.8 Filter Size Extremes: $F = 1$ vs $F = n$

$F = 1$: no spatial mixing, output size unchanged



$F = n$: filter covers whole image, output collapses



$F = 1$: every output pixel is a linear recombination *across channels only*, at that exact spatial location, and the output keeps the input's spatial size. $F = n$ (no padding): only one valid window position exists, so the entire image collapses to one number per filter — this is really a fully-connected layer over the whole image in disguise, and all spatial/local information is compressed away in a single step.

2 Practice Questions

Q1 (PYQ-style, CS6510-3 Q1a / CS6510-4 Q1A — asked twice, high priority). Given a filter of width F and depth d , applied to an image of dimension (W_1, H_1) , derive the equation for (W_2, H_2) assuming S strides and P padding. If there are K kernels, what is the number of parameters formed?

Q2 (PYQ-style, CS6510-3 Q3b, reworded as direct question). An input image I of size $n \times n$ is convolved once with a filter of size 1×1 , and separately with a filter of size $n \times n$ (stride 1, no padding, in both cases). What happens to the output in each case, and why?

Q3 (PYQ-style, CS6510-5 Q5a). For the architecture $\text{INPUT}(128 \times 128 \times 3) \rightarrow \text{CONV-9-32} \rightarrow \text{POOL-2} \rightarrow \text{CONV-5-64} \rightarrow \text{POOL-2} \rightarrow \text{CONV-5-64} \rightarrow \text{POOL-2} \rightarrow \text{FC-10} \rightarrow \text{FC-3}$ (CONV- F - K = filter size $F \times F$, K filters, stride 1, no padding; POOL-2 = 2×2 , stride 2), compute the feature map size and parameter count at each layer.

Q4 (PYQ-style, CS6510-5 Q6a). A 5×5 input

$$\begin{pmatrix} 1 & 2 & 0 & 1 & 2 \\ 0 & 1 & 2 & 0 & 1 \\ 1 & 0 & 1 & 2 & 0 \\ 2 & 1 & 0 & 1 & 1 \\ 0 & 2 & 1 & 0 & 2 \end{pmatrix}$$

is convolved (stride 1, no padding) with the 3×3 filter $\begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$. Find the convolution output.

Then apply 2×2 max pooling with stride 1 to that output. Finally, what padding size would make the *convolution* output the same size as the original 5×5 input?

3 Answers

A1. Padding adds P pixels on each side, so the effective input size is $W_1 + 2P$ (this is exactly what §1.4's diagram shows). A filter of size F sliding with stride S fits at position 0, then every subsequent position consumes S more of the effective width, until it no longer fits — giving $\lfloor (W_1 + 2P - F)/S \rfloor + 1$ valid positions:

$$W_2 = \frac{W_1 - F + 2P}{S} + 1, \quad H_2 = \frac{H_1 - F + 2P}{S} + 1$$

Each of the K filters has $F \times F \times d$ weights plus 1 bias (from §1.6), and filters don't share parameters with each other:

$$\# \text{params} = K(F^2 d + 1)$$

A2. With $F = 1$ (§1.7 diagram, top): $W_2 = (n - 1 + 0)/1 + 1 = n$ — output is the same $n \times n$ size; each output pixel is a linear recombination of the input channels at that exact pixel location, with zero spatial mixing. With $F = n$ (§1.7 diagram, bottom): $W_2 = (n - n + 0)/1 + 1 = 1$ — the filter covers the entire image at once, exactly one valid position exists, so the output collapses to a single 1×1 value per filter: the convolution degenerates into a fully-connected layer over the whole image, discarding all spatial/local structure in one step.

A3. Using $W_2 = (W_1 - F)/S + 1$ with $S = 1, P = 0$ for conv layers, and halving for each 2×2 /stride-2 pool:

Layer	Output size	Parameters
INPUT	$128 \times 128 \times 3$	—
CONV-9-32	$(128 - 9 + 1) = 120 \rightarrow 120 \times 120 \times 32$	$32(9^2 \cdot 3 + 1) = 32(244) = 7808$
POOL-2	$60 \times 60 \times 32$	0
CONV-5-64	$(60 - 5 + 1) = 56 \rightarrow 56 \times 56 \times 64$	$64(5^2 \cdot 32 + 1) = 64(801) = 51264$
POOL-2	$28 \times 28 \times 64$	0
CONV-5-64	$(28 - 5 + 1) = 24 \rightarrow 24 \times 24 \times 64$	$64(5^2 \cdot 64 + 1) = 64(1601) = 102464$
POOL-2	$12 \times 12 \times 64$	0
FC-10	flatten $12 \cdot 12 \cdot 64 = 9216 \rightarrow 10$	$9216(10) + 10 = 92170$
FC-3	$10 \rightarrow 3$	$10(3) + 3 = 33$

Total parameters = $7808 + 51264 + 102464 + 92170 + 33 = 253,739$.

A4. Sliding the 3×3 filter over the 5×5 input (stride 1, no padding) gives a 3×3 output (from the formula: $(5 - 3)/1 + 1 = 3$). The very first window position (top-left, as pictured in §1.2) is

$$\begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 2 \\ 1 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix} = (1-0-0) + (0-0-2) + (1-0-1) = 1 - 2 + 0 = -1$$

matching the -1 shown in the diagram. Repeating this dot product at all 9 positions:

$$\text{Conv output} = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 1 \\ 1 & 0 & -1 \end{pmatrix}$$

Applying 2×2 max pooling with stride 1 (output size $(3 - 2)/1 + 1 = 2$), taking the max of

each 2×2 window (as in §1.8's diagram):

$$\text{top-left: } \max(-1, 0, 0, -1) = 0, \quad \text{top-right: } \max(0, 0, -1, 1) = 1$$

$$\text{bottom-left: } \max(0, -1, 1, 0) = 1, \quad \text{bottom-right: } \max(-1, 1, 0, -1) = 1$$

$$\text{Max-pooled output} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$$

To keep the convolution output the same size as the 5×5 input (i.e. "same" padding, stride 1, as in §1.4): $P = (F - 1)/2 = (3 - 1)/2 = \boxed{1}$.